

LAPORAN PROYEK AKHIR TEKNOLOGI DAN PEMROGRAMAN MOBILE

**LAPOR.IN: LAPOR KERUSAKAN INFRASTRUKTUR
SECARA ONLINE**



Disusun oleh:

Nama : Annas Sovianto

NIM : 123220045

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" YOGYAKARTA
2025**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

TEKNOLOGI DAN PEMROGRAMAN MOBILE

Disusun oleh:

Nama	No. Mahasiswa	Tanda Tangan
Annas Sovianto	123220045	

telah dipresentasikan pada tanggal

Menyetujui,
Dosen Pengampu

Bagus Muhammad Akbar, S.S.T., M. Kom.

KATA PENGANTAR

Segala puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan Proyek Akhir untuk mata kuliah Teknologi dan Pemrograman Mobile dengan judul "*Lapor.In: Lapor Kerusakan Infrastruktur Secara Online*".

Laporan ini berisi dokumentasi lengkap dari proses perancangan, implementasi, hingga pengujian aplikasi mobile *Lapor.In*, sebuah platform pengaduan masyarakat berbasis Flutter yang dilengkapi fitur pelaporan kerusakan infrastruktur secara cepat, transparan, dan berbasis lokasi. Aplikasi ini juga menggunakan database lokal Hive, integrasi API eksternal seperti Google Maps, serta berbagai layanan pendukung untuk meningkatkan fungsionalitas dan kenyamanan pengguna.

Penyusunan laporan ini tidak terlepas dari bantuan dan dukungan berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Bagus Muhammad Akbar, S.S.T., M.Kom., selaku dosen pengampu mata kuliah Teknologi dan Pemrograman Mobile yang telah memberikan arahan dan motivasi selama proses pengerjaan proyek.
2. Seluruh rekan dan pihak yang telah memberikan masukan, bantuan teknis, serta semangat selama pengembangan aplikasi ini berlangsung.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, saran dan kritik yang membangun sangat penulis harapkan demi perbaikan di masa mendatang.

Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menjadi referensi dalam pengembangan aplikasi mobile berbasis Flutter, khususnya dalam konteks pelayanan publik dan pengaduan masyarakat.

Bantul, 9 Juni 2025

Penulis

DAFTAR ISI

HALAMAN JUDUL	i
HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN DAN PERANCANGAN.....	1
1.1 Latar Belakang	1
1.2 Deskripsi Sistem.....	2
1.3 Analisis Kebutuhan	3
1.4 Spesifikasi Teknis.....	4
1.5 Perancangan Sistem.....	5
BAB II IMPLEMENTASI	10
2.1 Deskripsi Umum Implementasi.....	10
2.2 Struktur Proyek.....	10
2.3 Penjelasan Code	11
2.4 Penggunaan API.....	22
2.5 Tampilan Program.....	25
BAB III PENGUJIAN	30
3.1 Metode Testing.....	30
3.2 Rencana Pengujian	30
3.3 Tabel Hasil Pengujian	31
BAB IV PENUTUP.....	35
5.1 Kesimpulan.....	35
5.2 Saran.....	35
DAFTAR PUSTAKA.....	37
LAMPIRAN	38

BAB I

PENDAHULUAN DAN PERANCANGAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi telah membawa perubahan signifikan dalam berbagai aspek kehidupan masyarakat, termasuk dalam cara masyarakat berinteraksi dengan pemerintah dan menyampaikan aspirasi mereka. Sistem pengaduan masyarakat yang efektif dan responsif menjadi semakin penting dalam mewujudkan tata pemerintahan yang baik (good governance) dan meningkatkan partisipasi publik dalam pembangunan.

Sistem pengaduan tradisional seringkali memiliki berbagai keterbatasan, seperti proses yang lambat, kurangnya transparansi, dan kesulitan dalam memantau tindak lanjut pengaduan. Hal ini dapat menyebabkan ketidakpuasan masyarakat dan menurunkan kepercayaan terhadap pemerintah.

Dalam konteks ini, aplikasi mobile "Lapor.In" hadir sebagai solusi inovatif untuk mengatasi permasalahan tersebut. "Lapor.In" adalah platform pengaduan masyarakat berbasis mobile yang memungkinkan masyarakat untuk menyampaikan laporan atau keluhan dengan mudah, cepat, dan transparan. Aplikasi ini memanfaatkan teknologi Flutter untuk menciptakan antarmuka pengguna yang menarik dan responsif, serta penyimpanan data lokal Hive untuk memastikan ketersediaan data dan kinerja aplikasi yang optimal.

"Lapor.In" dirancang untuk menjadi jembatan antara masyarakat dan pemerintah, memfasilitasi komunikasi dua arah yang efektif, dan mendorong penyelesaian masalah yang lebih cepat dan efisien. Dengan fitur-fitur seperti pelaporan berbasis lokasi, notifikasi status pengaduan, dan integrasi dengan sistem pemerintah yang ada, "Lapor.In" diharapkan dapat meningkatkan kualitas pelayanan publik, memperkuat partisipasi masyarakat, dan mewujudkan tata pemerintahan yang lebih baik.

Pengembangan aplikasi "Lapor.In" juga sejalan dengan upaya pemerintah dalam mendorong digitalisasi pelayanan publik dan meningkatkan kualitas hidup masyarakat melalui pemanfaatan teknologi informasi. Dengan demikian, laporan ini akan membahas secara rinci proses pengembangan aplikasi "Lapor.In", mulai dari analisis kebutuhan, perancangan sistem, implementasi, pengujian, hingga evaluasi, serta potensi dampaknya terhadap masyarakat dan pemerintah..

1.2 Deskripsi Sistem

Aplikasi "Lapor.In" adalah sebuah sistem pengaduan masyarakat berbasis mobile yang dirancang untuk memfasilitasi penyampaian laporan atau keluhan dari masyarakat kepada pihak berwenang secara lebih efisien dan transparan. Sistem ini terdiri dari beberapa komponen utama:

1. Aplikasi Mobile (Flutter): Aplikasi mobile adalah antarmuka utama yang digunakan oleh masyarakat untuk membuat dan mengirimkan laporan. Aplikasi ini menyediakan fitur-fitur berikut:
 - a. Formulir Pengaduan: Memungkinkan pengguna untuk mengisi detail laporan, termasuk deskripsi masalah, lokasi kejadian, kategori laporan, dan lampiran foto atau video.
 - b. Pelacakan Status: Memungkinkan pengguna untuk memantau status laporan mereka, mulai dari pengajuan hingga penyelesaian.
 - c. Notifikasi: Memberikan pemberitahuan kepada pengguna tentang perkembangan laporan mereka.
 - d. Profil Pengguna: Memungkinkan pengguna untuk mengelola informasi pribadi dan melihat riwayat laporan mereka.
2. Penyimpanan Data Lokal (Hive): Aplikasi ini menggunakan Hive sebagai database lokal untuk menyimpan data-data penting seperti:
 - a. Informasi Pengguna: Menyimpan data pengguna seperti preferensi dan informasi akun.
 - b. Notifikasi: Menyimpan data notifikasi yang diterima pengguna.
 - c. Status Laporan: Menyimpan status laporan yang pernah diajukan.
3. Layanan Pendukung: Aplikasi ini juga terintegrasi dengan beberapa layanan pendukung untuk meningkatkan fungsionalitasnya:
 - a. Currency Service: Untuk mengelola dan menampilkan informasi nilai tukar mata uang (jika ada fitur donasi atau pembayaran).
 - b. Notification Service: Untuk mengirimkan notifikasi kepada pengguna melalui berbagai saluran (misalnya, push notifications).
 - c. Layanan API Eksternal: Berinteraksi dengan API eksternal untuk fitur-fitur seperti geolokasi (misalnya, Google Maps API) dan pengiriman laporan ke sistem pemerintah yang ada.

Sistem "Lapor.In" dirancang dengan arsitektur modular yang memungkinkan pengembangan dan pemeliharaan yang lebih mudah. Setiap komponen sistem memiliki tanggung jawab yang jelas dan dapat dikembangkan secara independen. Sistem ini juga dirancang untuk menjadi responsif dan mudah digunakan, sehingga dapat diakses oleh masyarakat dari berbagai kalangan.

1.3 Analisis Kebutuhan

Analisis kebutuhan dilakukan untuk memahami fitur dan karakteristik yang diperlukan agar aplikasi "Lapor.In" dapat memenuhi tujuan yang telah ditetapkan. Analisis ini dibagi menjadi dua kategori utama: kebutuhan fungsional dan kebutuhan non-fungsional.

1. Kebutuhan Fungsional

Kebutuhan fungsional menjelaskan fungsi-fungsi yang harus dapat dilakukan oleh sistem. Berikut adalah beberapa kebutuhan fungsional utama untuk aplikasi "Lapor.In":

- a. Autentikasi Pengguna: Sistem harus memungkinkan pengguna untuk mendaftar (registrasi) dan masuk (login) dengan aman.
- b. Pengajuan Laporan: Sistem harus menyediakan formulir yang mudah digunakan untuk membuat dan mengirimkan laporan, termasuk deskripsi masalah, lokasi kejadian, kategori laporan, dan lampiran media (foto/video).
- c. Pelacakan Laporan: Sistem harus memungkinkan pengguna untuk melacak status laporan mereka secara real-time.
- d. Notifikasi: Sistem harus mengirimkan notifikasi kepada pengguna tentang perkembangan laporan mereka (misalnya, laporan diterima, sedang diproses, selesai).
- e. Manajemen Profil: Sistem harus memungkinkan pengguna untuk mengelola informasi profil mereka.
- f. Pencarian Laporan: Sistem harus menyediakan fitur pencarian untuk memudahkan pengguna mencari laporan berdasarkan kata kunci, kategori, atau lokasi.
- g. Penyimpanan Data Lokal: Sistem harus dapat menyimpan data-data penting seperti informasi pengguna, notifikasi, dan status laporan secara lokal menggunakan Hive.
- h. Integrasi API: Sistem harus dapat berintegrasi dengan API eksternal untuk fitur-fitur seperti geolokasi (misalnya, Google Maps API) dan pengiriman laporan ke sistem pemerintah yang ada.

2. Kebutuhan Non-Fungsional

Kebutuhan non-fungsional menjelaskan kualitas dan batasan sistem. Berikut adalah beberapa kebutuhan non-fungsional utama untuk aplikasi "Lapor.In":

- a. Kinerja: Aplikasi harus responsif dan memiliki waktu muat yang cepat.
- b. Keamanan: Sistem harus aman dan melindungi data pengguna dari akses yang tidak sah.
- c. Ketersediaan: Aplikasi harus tersedia dan dapat diakses oleh pengguna setiap saat (24/7).
- d. Skalabilitas: Sistem harus dapat menangani peningkatan jumlah pengguna dan laporan tanpa mengurangi kinerja.
- e. Usabilitas: Aplikasi harus mudah digunakan dan memiliki antarmuka yang intuitif.
- f. Portabilitas: Aplikasi harus dapat berjalan di berbagai perangkat mobile dengan sistem operasi Android dan iOS.
- g. Pemeliharaan: Kode aplikasi harus mudah dipelihara dan dimodifikasi.

1.4 Spesifikasi Teknis

Aplikasi "Lapor.In" dikembangkan dengan menggunakan teknologi dan spesifikasi teknis sebagai berikut:

1. Bahasa Pemrograman: Dart
2. Framework: Flutter
3. IDE: Visual Studio Code
4. Manajemen Paket: Pub (Dart Package Manager)
5. Penyimpanan Data Lokal: Hive (NoSQL database)
6. Arsitektur: Modular Architecture
7. UI Library: Flutter Material Components
8. State Management: Provider (atau dapat disesuaikan dengan kebutuhan proyek)
9. HTTP Client: http package (untuk komunikasi dengan API eksternal)
10. Asynchronous Programming: async/await (untuk penanganan operasi asinkron)
11. Version Control: Git
12. Platform: Android dan iOS
13. Minimum SDK Version: Android API 21 (Android 5.0 Lollipop), iOS 11
14. Layanan Pendukung:
Currency Service (jika ada fitur donasi atau pembayaran)

Notification Service (Firebase Cloud Messaging atau alternatif lainnya)

API Eksternal (misalnya, Google Maps API untuk geolokasi)

15. Tools Pendukung:

Logger (misalnya, logger package untuk logging aplikasi)

Lintar (untuk memastikan kualitas kode)

16. Dependencies:

hive_flutter: Untuk integrasi Hive dengan Flutter

http: Untuk melakukan HTTP request ke API

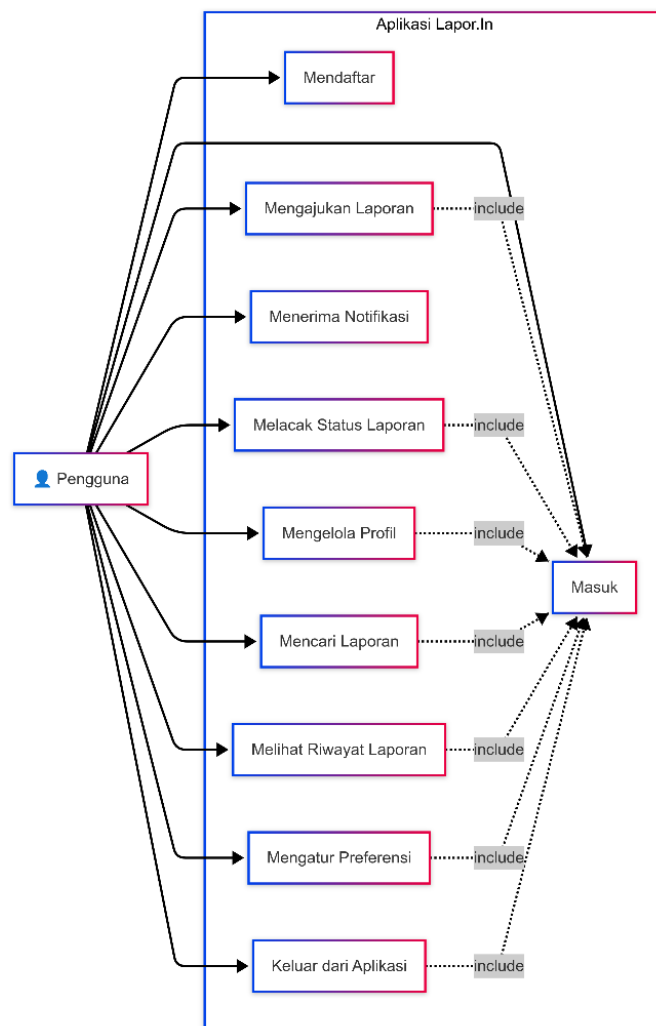
provider: Untuk manajemen state

intl: Untuk format tanggal dan angka

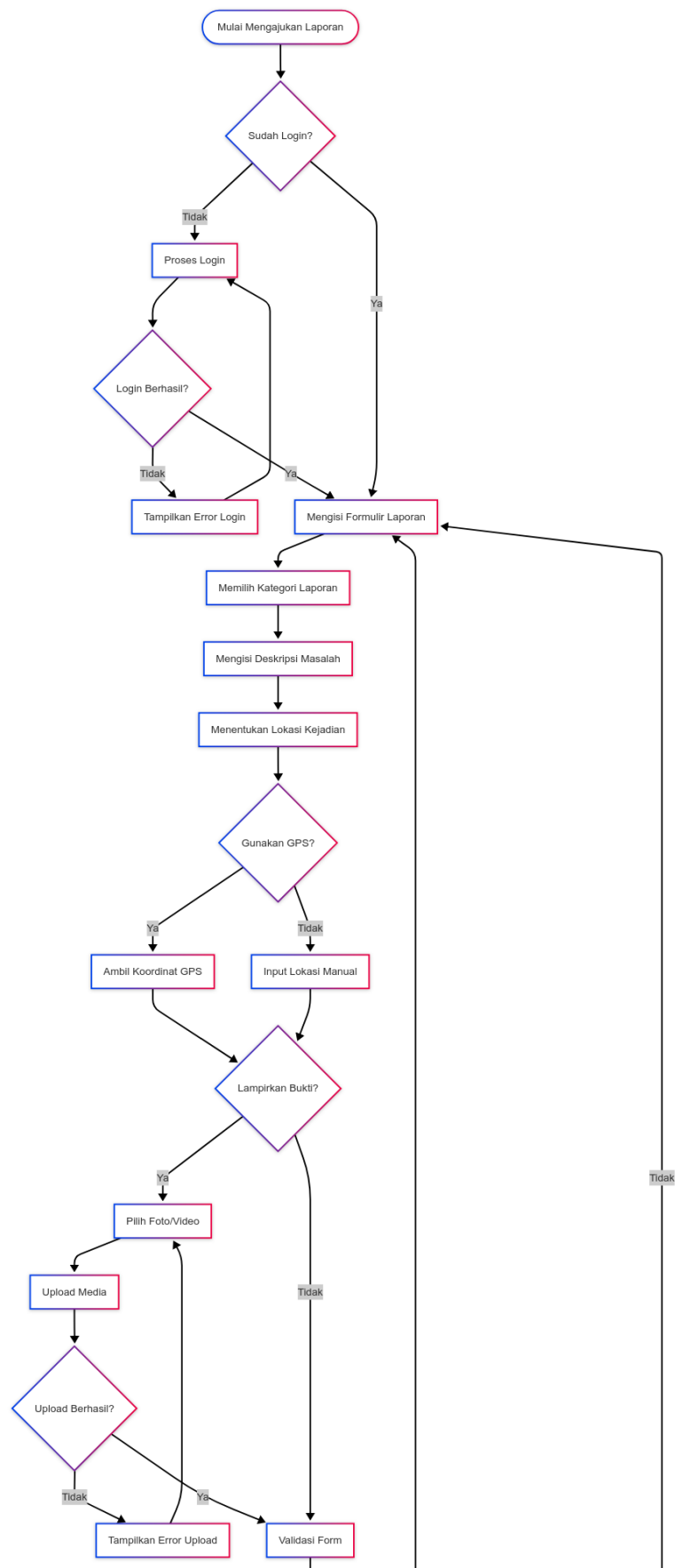
shared_preferences: Untuk menyimpan data preferensi pengguna

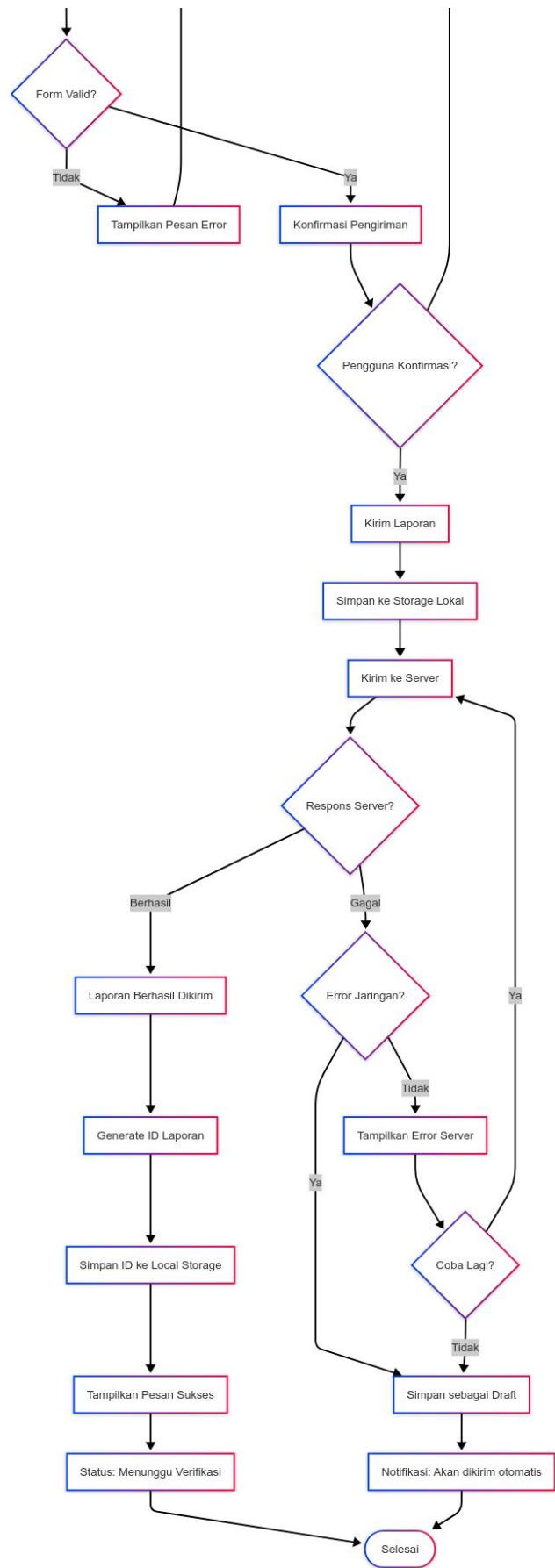
1.5 Perancangan Sistem

1 Use Case Diagram

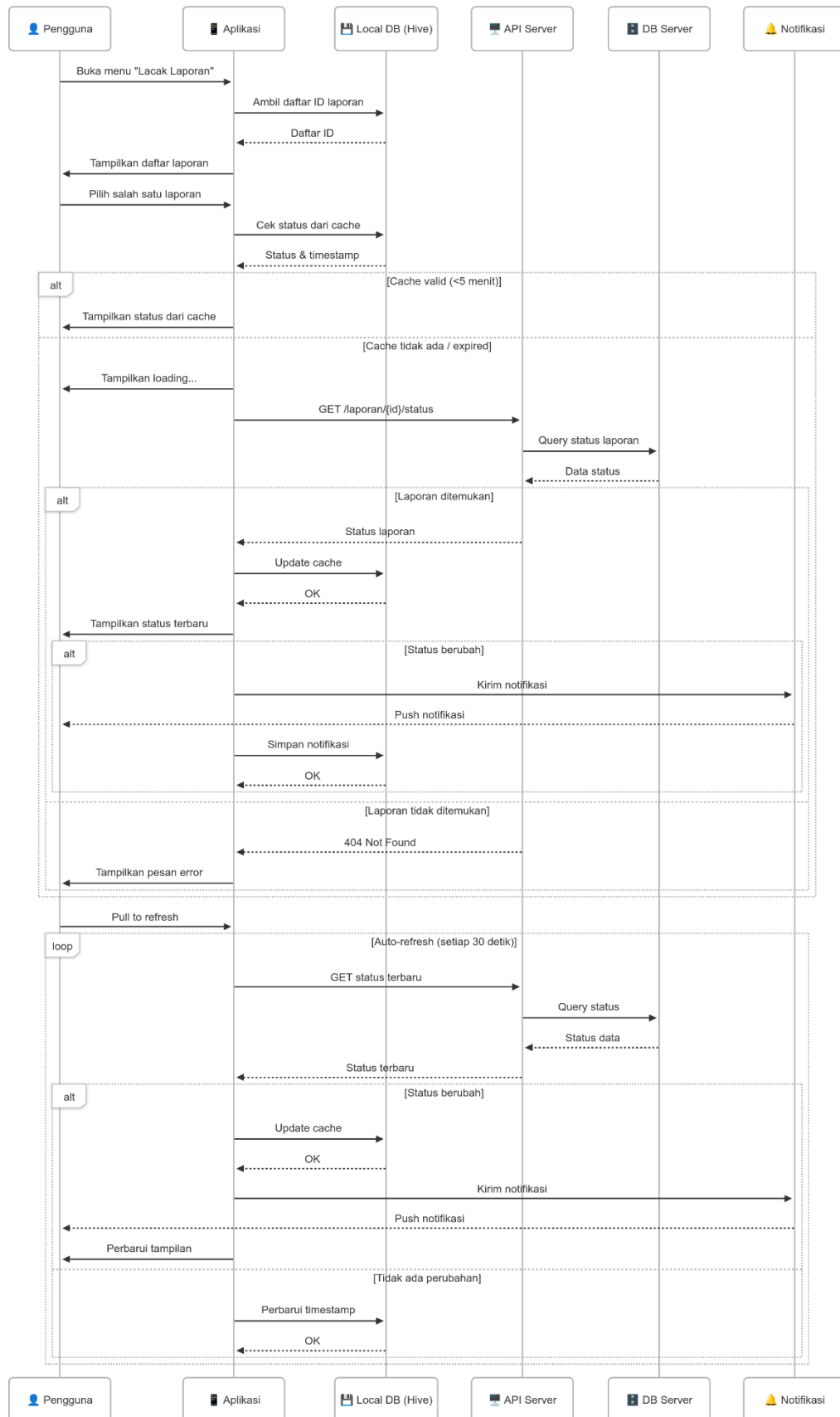


2 Activity Diagram

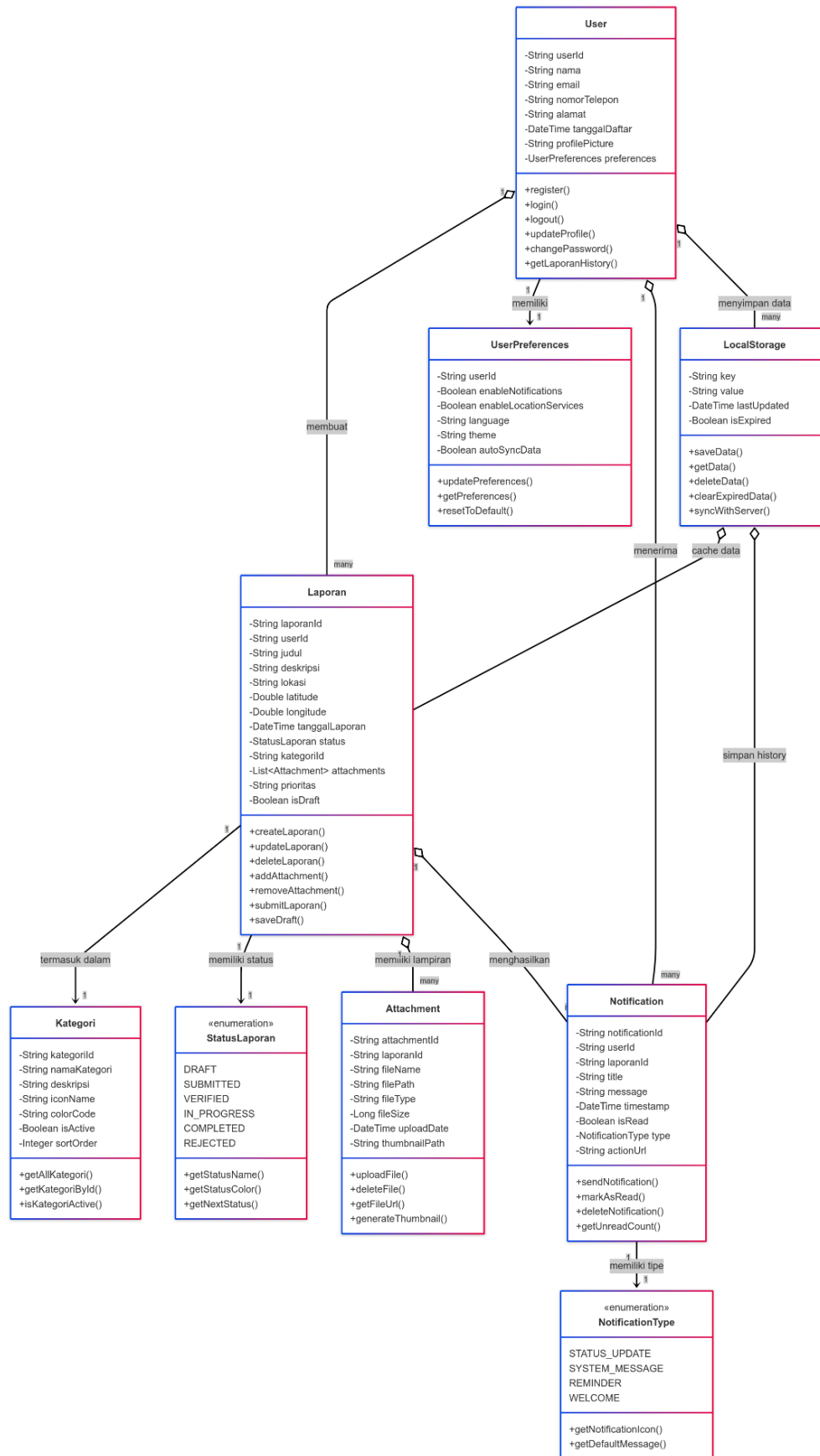




3 Sequence Diagram



4 Class Diagram



BAB II

IMPLEMENTASI

2.1 Deskripsi Umum Implementasi

Implementasi aplikasi "Lapor.In" melibatkan penerjemahan desain sistem dan spesifikasi teknis ke dalam kode program yang berfungsi. Proses ini mencakup penulisan kode dalam bahasa Dart menggunakan framework Flutter, konfigurasi database lokal Hive, integrasi dengan API eksternal, dan pengujian unit serta integrasi. Implementasi dilakukan dengan pendekatan modular, di mana setiap fitur dan komponen sistem dikembangkan secara terpisah dan kemudian diintegrasikan untuk membentuk aplikasi yang lengkap. Fokus utama dalam implementasi adalah menciptakan aplikasi yang responsif, mudah digunakan, aman, dan sesuai dengan kebutuhan pengguna. Proses implementasi juga mencakup penanganan error, logging, dan optimasi kinerja untuk memastikan aplikasi berjalan dengan baik di berbagai perangkat dan kondisi jaringan.

2.2 Struktur Proyek

Struktur proyek aplikasi "Lapor.In" di dalam folder lib diatur sedemikian rupa untuk memisahkan berbagai komponen sistem dan memfasilitasi pengembangan, pemeliharaan, dan pengujian. Berikut adalah penjelasan mengenai struktur folder dan file utama:

1. `main.dart`: File ini adalah titik masuk utama (entry point) dari aplikasi Flutter. Di sini, aplikasi diinisialisasi, konfigurasi awal dilakukan (misalnya, inisialisasi Hive, pengaturan tema), dan widget utama (MyApp) dijalankan.
2. `models`: Folder ini berisi definisi kelas model data yang digunakan dalam aplikasi. Contohnya:
 - a. `user_credit.dart`: Model untuk data kredit pengguna (jika ada fitur donasi atau pembayaran).
 - b. `notification.dart`: Model untuk data notifikasi.
 - c. `report.dart`: Model untuk data laporan pengaduan.
 - d. `category.dart`: Model untuk data kategori laporan.
3. `services`: Folder ini berisi kelas-kelas yang menyediakan layanan atau fungsi-fungsi tertentu yang digunakan oleh aplikasi. Contohnya:
 - a. `auth_service.dart`: Layanan untuk autentikasi pengguna (registrasi, login, logout).
 - b. `report_service.dart`: Layanan untuk mengelola laporan (mengirim, mengambil, memperbarui status).

- c. `category_service.dart`: Layanan untuk mengelola kategori laporan.
 - d. `currency_service.dart`: Layanan untuk mendapatkan nilai tukar mata uang (jika ada fitur donasi atau pembayaran).
 - e. `notification_service.dart`: Layanan untuk mengelola notifikasi.
4. `utils`: Folder ini berisi utilitas atau helper functions yang digunakan di seluruh aplikasi. Contohnya:
- a. `hive_box.dart`: Definisi nama-nama box Hive yang digunakan untuk penyimpanan data lokal.
 - b. `constants.dart`: Konstanta-konstanta yang digunakan di seluruh aplikasi (misalnya, URL API, warna default).
 - c. `validators.dart`: Fungsi-fungsi validasi untuk input pengguna.
5. `views`: Folder ini berisi widget-widget yang membentuk antarmuka pengguna (UI) aplikasi. Struktur folder di dalam views dapat diatur berdasarkan fitur atau modul aplikasi. Contohnya:
- a. `auth_wrapper.dart`: Widget yang menentukan tampilan awal aplikasi berdasarkan status autentikasi pengguna.
 - b. `auth/`: Folder yang berisi widget-widget terkait autentikasi (misalnya, `login_page.dart`, `register_page.dart`).
 - c. `home/`: Folder yang berisi widget-widget untuk halaman utama aplikasi.
 - d. `report/`: Folder yang berisi widget-widget terkait laporan (misalnya, `create_report_page.dart`, `report_detail_page.dart`, `report_list_page.dart`).
 - e. `profile/`: Folder yang berisi widget-widget terkait profil pengguna.
 - f. Struktur proyek ini dirancang untuk memisahkan logika bisnis, data, dan tampilan, sehingga memudahkan pengembangan, pengujian, dan pemeliharaan aplikasi.

2.3 Penjelasan Code

1. Implementasi Login dan Registrasi dengan Enkripsi

```
// filepath: lib/services/auth_service.dart
...existing code...
// Login user
static Future<User> login({
  required String phoneNumber,
  required String password,
}) async {
  _logger.i('Starting user login for phone: $phoneNumber');

  try {
    final deviceInfo = await DeviceInfoHelper.getDeviceInfo();
```

```

_logger.d('Device info: $deviceInfo');

final data = {
  'phone_number': phoneNumber,
  'password': password,
  'device_info': deviceInfo,
};

final response = await BaseNetwork.post('/users/login', data);
_logger.d('Login response: $response');

// Handle different response structures
Map<String, dynamic> userData;
if (response.containsKey('user') && response['user'] != null)
{
  userData = response['user'];
} else if (response.containsKey('data') && response['data'] !=
null) {
  userData = response['data'];

} else {
  throw Exception('Invalid login response format');
}

final user = User.fromJson(userData);
_logger.i('User login successful for phone:
${user.phoneNumber}');

// Store phone number for session management
final prefs = await SharedPreferences.getInstance();
await prefs.setString(_phoneNumberKey, user.phoneNumber!);

return user;
} catch (e) {
  _logger.e('User login failed for phone: $phoneNumber - Error:
$e');
  rethrow;
}
}

// Check user session
static Future<bool> isLoggedIn() async {
  final prefs = await SharedPreferences.getInstance();
  final phoneNumber = prefs.getString(_phoneNumberKey);

  if (phoneNumber == null) {
    _logger.w('No phone number found in local storage');
    return false;
  }

  _logger.i('Checking session for phone: $phoneNumber');
  try {
    final response = await BaseNetwork.post(
      '/users/session',

```



```

        {'phone_number': phoneNumber},
    );
    final isLoggedIn = response['isLoggedIn'] ?? false;
    _logger.i(
        'Session check result for phone: $phoneNumber - isLoggedIn: $isLoggedIn',
    );
    return isLoggedIn;
  } catch (e) {
    _logger.e('Session check failed for phone: $phoneNumber - Error: $e');
    return false;
  }
}

// Logout user
static Future<void> logout() async {
  _logger.i('Starting user logout');

  final prefs = await SharedPreferences.getInstance();
  final phoneNumber = prefs.getString(_phoneNumberKey);

  if (phoneNumber != null) {
    _logger.d('Logging out user with phone: $phoneNumber');
    try {
      await BaseNetwork.post('/users/logout', {'phone_number':
phoneNumber});
      _logger.i('Server logout successful for phone: $phoneNumber');
    } catch (e) {
      _logger.w(
        'Server logout failed for phone: $phoneNumber - Error: $e,
continuing with local logout',
      );
    }
  } else {
    _logger.w(
      'No phone number found, skipping server logout and clearing
local session',
    );
  }

  // Clear local session
  await prefs.remove(_phoneNumberKey);
  _logger.i('Local session cleared');
}

...existing code...

```

Penjelasan:

Kode ini menunjukkan implementasi login dan logout pengguna. Proses login melibatkan pengiriman nomor telepon dan kata sandi ke server melalui API (/users/login). Setelah berhasil, nomor telepon pengguna disimpan di SharedPreferences untuk manajemen sesi. Fungsi isLoggedIn memeriksa keberadaan nomor telepon di SharedPreferences dan

melakukan verifikasi sesi dengan server melalui API (/users/session). Proses logout menghapus nomor telepon dari SharedPreferences dan memberitahu server melalui API (/users/logout).

2. Koneksi Database / Penyimpanan (Hive)

```
// filepath: lib/main.dart
...existing code...
// Initialize Hive
await Hive.initFlutter();
Hive.registerAdapter(UserCreditAdapter());
Hive.registerAdapter(AppNotificationAdapter());
await Hive.openBox<UserCredit>(HiveBox.userCredits);
await Hive.openBox<AppNotification>(HiveBox.notifications);
await Hive.openBox<Map>(HiveBox.reportStatuses);
...existing code...
```

Penjelasan:

Kode ini menunjukkan inisialisasi Hive sebagai penyimpanan data lokal. Hive.initFlutter() menginisialisasi Hive untuk digunakan dengan Flutter. Hive.registerAdapter() mendaftarkan adapter untuk kelas UserCredit dan AppNotification, memungkinkan Hive untuk menyimpan objek dari kelas-kelas ini. Hive.openBox() membuka box (semacam tabel) untuk menyimpan data UserCredit, AppNotification, dan status laporan.

3. Web Service / API dan Fasilitas LBS

```
// filepath: lib/services/report_service.dart
...existing code...
// Add this new method for searching public reports
static Future<List<PublicReport>> searchPublicReports(
  String query, {
  String? category,
  String? status,
  int limit = 20,
  int offset = 0,
}) async {
  _logger.i('Searching public reports with query: $query');

  if (query.trim().length < 2) {
    throw Exception('Query pencarian minimal 2 karakter');
  }

  try {
    // Build query string manually
    String queryString =
    'q=${Uri.encodeQueryComponent(query.trim())}';
    queryString += '&limit=$limit&offset=$offset';

    if (category != null && category.isNotEmpty) {
      queryString +=
      '&category=${Uri.encodeQueryComponent(category)}';
    }
  }
}
```

```

        if (status != null && status.isNotEmpty) {
            queryString
            '&status=${Uri.encodeQueryComponent(status)}';
        }

        final endpoint = '/public/reports/search?$queryString';
        final response = await BaseNetwork.get(endpoint);
        ...existing code...
    }
}

```

Penjelasan:

Kode ini menunjukkan penggunaan API untuk mencari laporan publik. Fungsi `searchPublicReports` mengirimkan permintaan GET ke endpoint `/public/reports/search` dengan parameter query seperti kata kunci pencarian (query), kategori (category), dan status (status). Kode ini memanfaatkan LBS dengan memungkinkan pencarian laporan berdasarkan lokasi yang relevan dengan laporan tersebut.

4. Bottom Navigation, Menu Profil, dan Logout

```

// filepath: lib/views/navigation.dart
...existing code...
@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            backgroundColor: Colors.blue.shade700,
            foregroundColor: Colors.white,
            elevation: 0,
            // Using light status bar icons with transparent background
            systemOverlayStyle: SystemUiOverlayStyle.light.copyWith(
                statusBarColor: Colors.transparent,
            ),
            title: Text(
                _pageTitles[_currentIndex],
                style: const TextStyle(
                    color: Colors.white,
                    fontWeight: FontWeight.bold,
                ),
            ),
        ),
        actions: [
            IconButton(
                onPressed: () {
                    Navigator.of(context).push(
                        MaterialPageRoute(
                            pageBuilder:
                                (context, animation, secondaryAnimation) =>
                                    const ProfilePage(),
                            transitionDuration: Duration.zero,
                            reverseTransitionDuration: Duration.zero,
                        ),
                    );
                },
                icon: const Icon(Icons.person, color: Colors.white),
            ),
        ],
    );
}

```

```

        tooltip: 'Profile',
      ),
    ],
  ),
  body: _pages[_currentIndex],
  bottomNavigationBar: Container(
    height: 80,
    decoration: BoxDecoration(
      color: Colors.white,
      boxShadow: [
        BoxShadow(
          color: Colors.grey.withValues(alpha:0.1),
          blurRadius: 10,
          offset: const Offset(0, -2),
        ),
      ],
    ),
  ),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      _buildNavigationItem(
        icon: Icons.home_outlined,
        label: 'Home',
        index: 0,
      ),
      _buildNavigationItem(
        icon: Icons.report_outlined,
        label: 'Lapor',
        index: 1,
      ),
      _buildNavigationItem(
        icon: Icons.history,
        label: 'History',
        index: 2,
      ),
      _buildNavigationItem(
        icon: Icons.notifications_outlined,
        label: 'Pesan',
        index: 3,
        showBadge: true,
        badgeCount: _unreadNotificationCount,
      ),
    ],
  ),
);
}
...existing code...

```

Penjelasan:

Kode ini menunjukkan implementasi bottom navigation bar dengan ikon-ikon untuk Home, Lapor, History, dan Notifikasi. Terdapat juga ikon profil di app bar yang mengarah ke halaman profil. Di halaman profil, terdapat tombol "Logout" yang memanggil fungsi

AuthService.logout() dan mengarahkan pengguna kembali ke halaman pendaratan (LandingPage).

5. Konversi Mata Uang dan Waktu

```
// filepath: lib/views/lapor/lapor_page.dart
...existing code...
Future<void> _changeCurrency() async {
  final currencies = ['IDR', 'USD', 'JPY', 'CNY'];

  final selectedCurrency = await showModalBottomSheet<String>(
    context: context,
    isScrollControlled: true,
    builder: (BuildContext context) {
      return Container(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const Text(
              'Select Currency',
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 16),
            ...currencies.map((currency) {
              return ListTile(
                title: Text(currency),
                onTap: () => Navigator.pop(context, currency),
              );
            }).toList(),
          ],
        ),
      );
    },
  );

  if (selectedCurrency != null &&
    selectedCurrency != _userCredit!.preferredCurrency) {
    await CreditService.updatePreferredCurrency(
      _userPhoneNumber!,
      selectedCurrency,
    );
    await _loadUserCreditData();
  }
}
...existing code...
```

Penjelasan:

Kode ini menunjukkan implementasi konversi mata uang. Fungsi `_changeCurrency` menampilkan bottom sheet yang memungkinkan pengguna memilih mata uang yang

diinginkan. Setelah pengguna memilih mata uang, fungsi `CreditService.updatePreferredCurrency` dipanggil untuk memperbarui preferensi mata uang pengguna.

6. Pencarian dengan Suggestion

```
// filepath: lib/views/home/home_page.dart
...existing code...
class _HomePageState extends State<HomePage> {
  final TextEditingController _searchController =
  TextEditingController();
  List<PublicReport> _searchResults = [];
  bool _isSearching = false;
  String _lastSearchQuery = '';

  @override
  void initState() {
    super.initState();
    _searchController.addListener(_onSearchChanged);
  }

  @override
  void dispose() {
    _searchController.removeListener(_onSearchChanged);
    _searchController.dispose();
    super.dispose();
  }

  Future<void> _onSearchChanged() async {
    final query = _searchController.text.trim();
    if (query.length < 2 && query.isNotEmpty) {
      // Don't search for queries less than 2 characters
      setState(() {
        _searchResults = [];
        _isSearching = false;
      });
      return;
    }

    if (query == _lastSearchQuery) return; // Prevent duplicate
    searches

    setState(() {
      _isSearching = query.isNotEmpty;
      _lastSearchQuery = query;
    });

    if (query.isEmpty) {
      setState(() {
        _searchResults = [];
        _isSearching = false;
      });
      return;
    }
  }
}
```

```

    _performSearch(query);
}

Future<void> _performSearch(String query) async {
  try {
    final results = await
ReportService.searchPublicReports(query);
    if (mounted) {
      setState(() {
        _searchResults = results;
        _isSearching = false;
      });
    }
  } catch (e) {
    if (mounted) {
      setState(() {
        _isSearching = false;
      });

      // Show error message to user if it's not a "no results"
case
      if (!e.toString().contains('No reports found') &&
        !e.toString().contains('Tidak ada aduan ditemukan'))
      {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text('Error searching reports:
${e.toString()}'),
            backgroundColor: Colors.red,
            duration: const Duration(seconds: 3),
          ),
        );
      }
    }
  }
}

void _clearSearch() {
  _searchController.clear();
  setState(() {
    _searchResults = [];
    _isSearching = false;
    _lastSearchQuery = '';
  });
}

...existing code...

```

Penjelasan:

Kode ini menunjukkan implementasi pencarian laporan di halaman utama. TextEditingController digunakan untuk memantau perubahan pada input pencarian. Fungsi _onSearchChanged dipanggil setiap kali teks pencarian berubah. Fungsi ini memanggil

ReportService.searchPublicReports untuk mendapatkan hasil pencarian dari API dan memperbarui tampilan dengan hasil yang baru.

7. Implementasi Pemilihan saat Dropdown Kategori

```
// filepath: lib/views/lapor/steps/step1_report_info.dart
...existing code...
class _Step1ReportInfoState extends State<Step1ReportInfo> {
  String? _selectedCategory;
  List<ReportCategory> _categories = [];

  @override
  void initState() {
    super.initState();
    _loadCategories();
  }

  Future<void> _loadCategories() async {
    try {
      final categories = await CategoryService.getCategories();
      setState(() {
        _categories = categories;
      });
    } catch (e) {
      // Handle error
      print('Error loading categories: $e');
    }
  }

  @override
  Widget build(BuildContext context) {
    return _buildDropdown<String>(
      labelText: 'Category',
      value: _selectedCategory,
      items: _categories.map((category) {
        return DropdownMenuItem<String>(
          value: category.id.toString(),
          child: Text(category.name),
        );
      }).toList(),
      onChanged: (value) {
        setState(() {
          _selectedCategory = value;
        });
        _updateData();
      },
      validator: (value) {
        if (value == null || value.isEmpty) {
          return 'Please select a category';
        }
        return null;
      },
    );
  }
}
```



```
...existing code...
```

Penjelasan:

Kode ini menunjukkan implementasi dropdown kategori di form laporan. Fungsi `_loadCategories` memanggil `CategoryService.getCategories` untuk mendapatkan daftar kategori dari API. Daftar kategori kemudian digunakan untuk membuat item-item dropdown. `_buildDropdown` (helper widget) digunakan untuk menampilkan dropdown dan memungkinkan pengguna memilih kategori.

8. Implementasi Sensor Accelerometer

```
// filepath: lib/services/notification_service.dart
...existing code...
static Future<void> createReportStatusNotification({
  required Report report,
  required String oldStatus,
  required String newStatus,
}) async {
  try {
    final notificationId =
DateTime.now().millisecondsSinceEpoch.toString();
    final timestamp = DateTime.now();

    // Get current user's phone number
    final currentUser = await AuthService.getCurrentUser();
    if (currentUser == null) {
      _logger.w('No current user found, cannot create
notification');
      return;
    }

    // Format status for display
    final oldStatusText = _formatStatusText(oldStatus);
    final newStatusText = _formatStatusText(newStatus);

    // Create notification object
    final notification = AppNotification(
      id: notificationId,
      title: 'Status Laporan Berubah',
      body: '${report.title} - $oldStatusText ke $newStatusText',
      reportTitle: report.title,
      oldStatus: oldStatus,
      newStatus: newStatus,
      reportId: report.id!,
      timestamp: timestamp,
      phoneNumber: currentUser.phoneNumber,
    );

    // Save to Hive
    final box =
Hive.openBox<AppNotification>(HiveBox.notifications);
    await box.put(notificationId, notification);
```

```

    _logger.i('Notification created and saved to Hive:
    ${notification.id}');

    // Show local notification
    await showNotification(
      int.parse(notificationId),
      notification.title,
      notification.body,
    );
  } catch (e) {
    _logger.e('Error creating report status notification: $e');
  }
}
...existing code...

```

Penjelasan:

Kode ini menunjukkan implementasi notifikasi perubahan status laporan. Fungsi `createReportStatusNotification` membuat objek notifikasi dan menyimpannya ke Hive. Fungsi ini juga memanggil `showNotification` untuk menampilkan notifikasi lokal kepada pengguna.

2.4 Penggunaan API

1. AuthService (#lib/services/auth_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk autentikasi pengguna, termasuk registrasi, login, logout, dan manajemen sesi.
- **Endpoint:**
 - `/users/register` (POST): Mendaftarkan pengguna baru.
 - `/users/login` (POST): Login pengguna.
 - `/users/check-session` (POST): Memeriksa sesi pengguna.
 - `/users/logout` (POST): Logout pengguna.
- **Analisis:** Service ini menggunakan `BaseNetwork` untuk melakukan permintaan HTTP ke API backend. Data pengguna disimpan secara lokal menggunakan `SharedPreferences`.

2. ReportService (#lib/services/report_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk mengelola laporan, termasuk mendapatkan statistik laporan, mendapatkan laporan berdasarkan tracking ID, mendapatkan detail laporan berdasarkan ID, dan mencari laporan publik.
- **Endpoint:**
 - `/public/statistics` (GET): Mendapatkan statistik laporan.

- /public/reports/track/{trackingId} (GET): Mendapatkan laporan berdasarkan tracking ID.
- /public/reports/{reportId} (GET): Mendapatkan detail laporan berdasarkan ID.
- /public/reports/search (GET): Mencari laporan publik.
- **Analisis:** Service ini menggunakan BaseNetwork untuk melakukan permintaan HTTP ke API backend.

3. CategoryService (#lib/services/category_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk mengelola kategori laporan.
- **Endpoint:**
 - /public/categories (GET): Mendapatkan daftar kategori laporan.
- **Analisis:** Service ini menggunakan BaseNetwork untuk melakukan permintaan HTTP ke API backend.

4. AgencyService (#lib/services/agency_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk mengelola daftar instansi pemerintah.
- **Endpoint:**
 - /public/agencies (GET): Mendapatkan daftar instansi pemerintah.
- **Analisis:** Service ini menggunakan BaseNetwork untuk melakukan permintaan HTTP ke API backend.

5. ReportSubmissionService (#lib/services/report_submission_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk mengirimkan laporan pengaduan.
- **Endpoint:**
 - /public/reports (POST Multipart): Mengirimkan laporan pengaduan dengan gambar.
- **Analisis:** Service ini menggunakan BaseNetwork untuk melakukan permintaan HTTP ke API backend.

6. LocationService (#lib/services/location_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk mendapatkan lokasi pengguna dan mendapatkan alamat dari koordinat.
- **Endpoint:**

- <https://maps.googleapis.com/maps/api/geocode/json> (GET): Mendapatkan alamat dari koordinat atau koordinat dari alamat menggunakan Google Maps API.
- **Analisis:** Service ini menggunakan Dio untuk melakukan permintaan HTTP ke Google Maps API. Service ini juga menggunakan Geolocator untuk mendapatkan lokasi pengguna.

7. NotificationService (#lib/services/notification_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk mengelola notifikasi, termasuk inisialisasi, menampilkan notifikasi lokal, dan memeriksa perubahan status laporan.
- **Fungsi Utama:**
 - `initialize()`: Menginisialisasi service notifikasi dan plugin `flutter_local_notifications`.
 - `startStatusPolling()`: Memulai polling periodik untuk memeriksa perubahan status laporan.
 - `_checkForStatusChanges()`: Memeriksa perubahan status laporan dengan membandingkan status yang tersimpan dengan status saat ini dari API.
 - `_createStatusChangeNotification()`: Membuat notifikasi jika ada perubahan status laporan.
 - `_showLocalNotification()`: Menampilkan notifikasi lokal menggunakan plugin `flutter_local_notifications`.
 - `getAllNotifications()`: Mengambil semua notifikasi untuk pengguna saat ini dari Hive.
- **Analisis:** Service ini menggunakan Hive untuk menyimpan notifikasi secara lokal dan `flutter_local_notifications` untuk menampilkan notifikasi lokal. Service ini juga menggunakan `ConnectivityService` untuk memeriksa koneksi internet sebelum memeriksa perubahan status laporan.

8. CurrencyService (#lib/services/currency_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk mendapatkan nilai tukar mata uang.
- **Endpoint:**
 - <https://api.exchangerate-api.com/v4/latest/IDR> (GET): Mendapatkan nilai tukar mata uang dari API eksternal.
- **Analisis:** Service ini menggunakan `http` package untuk melakukan permintaan HTTP ke API eksternal.

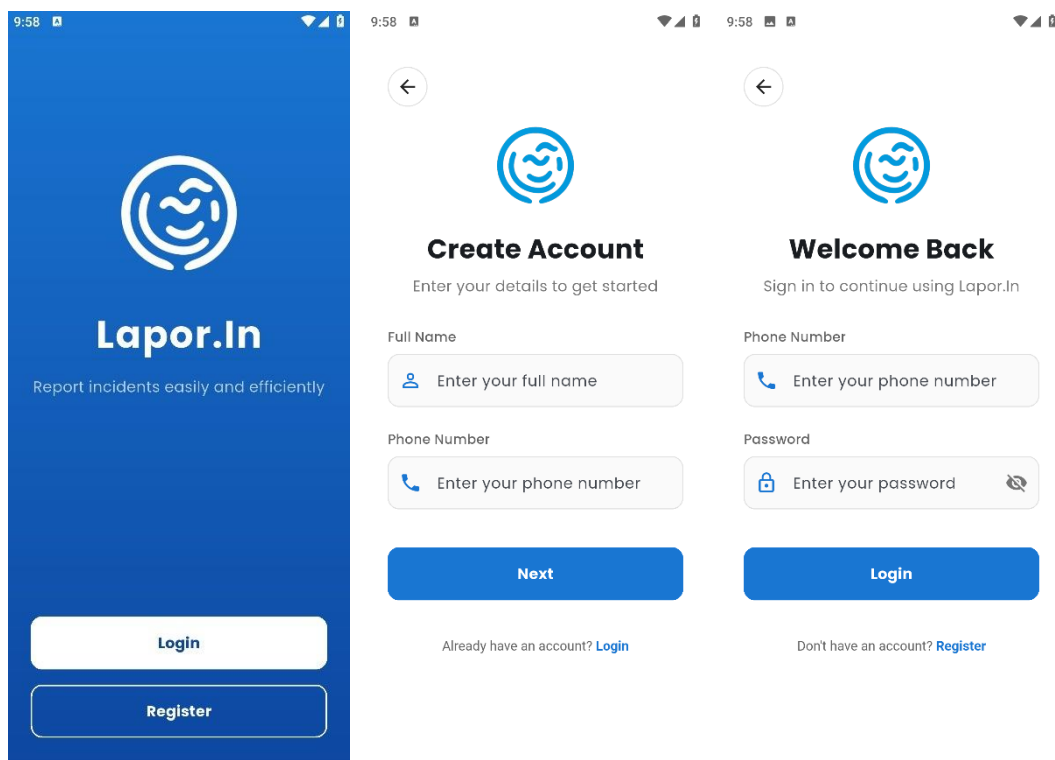
9. ConnectivityService (#lib/services/connectivity_service.dart)

- **Deskripsi:** Service ini bertanggung jawab untuk memeriksa koneksi internet.
- **Fungsi Utama:**
 - `hasConnection()`: Memeriksa apakah ada koneksi internet.
 - `onConnectivityChanged`: Stream yang memancarkan perubahan status koneksi internet.
- **Analisis:** Service ini menggunakan `connectivity_plus` package untuk memeriksa koneksi internet.

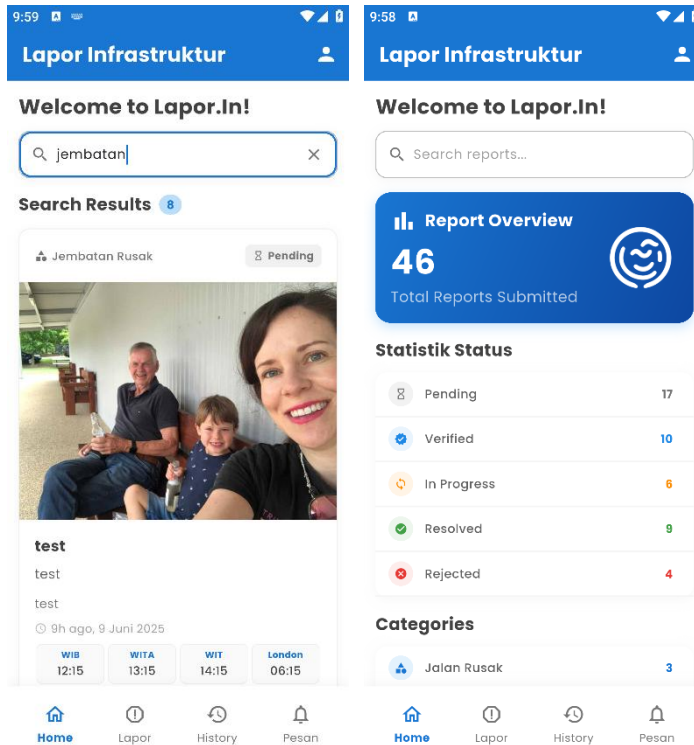
Dengan analisis ini, Anda dapat memahami bagaimana aplikasi "Lapor.In" menggunakan berbagai API dan services untuk menyediakan fungsionalitas yang lengkap kepada pengguna.

2.5 Tampilan Program

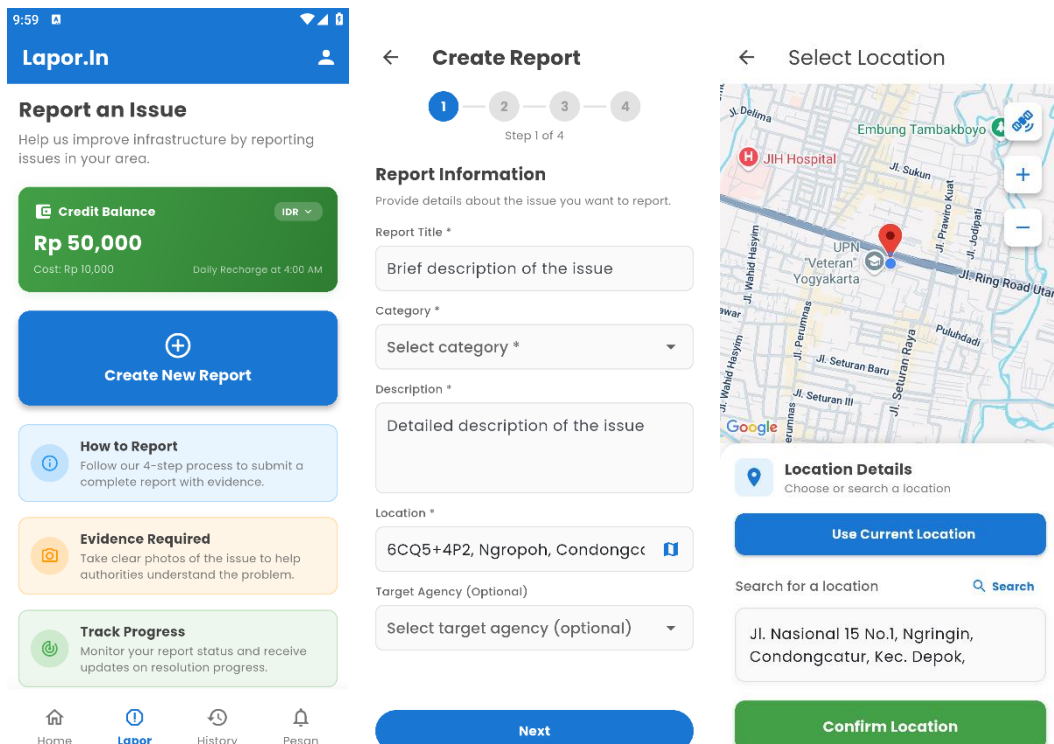
1. Login dan Register



2. Home dan Searching



3. Lapor dan Create New Report



10:00





Create Report

1

2

3

4

Step 4 of 4

Review & Submit

Please review your report before submitting.

1

Report Information



Shake Detected!

Do you want to submit your report now?

Cancel

Submit



Reporter Information

Name

: Annas Sovianto

Phone

: 12345



Evidence

Photo Evidence



12:00





Create Report

1

2

3

4

Step 4 of 4

Review & Submit

Please review your report before submitting.

1

Report Information



Report Submitted

Your report has been sent successfully.

OK

1

Reporter Information

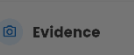
Name : Annas Sovianto

Phone : 12345

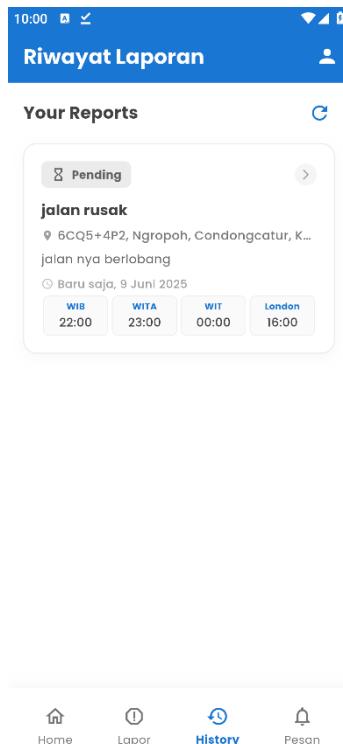
1

Evidence

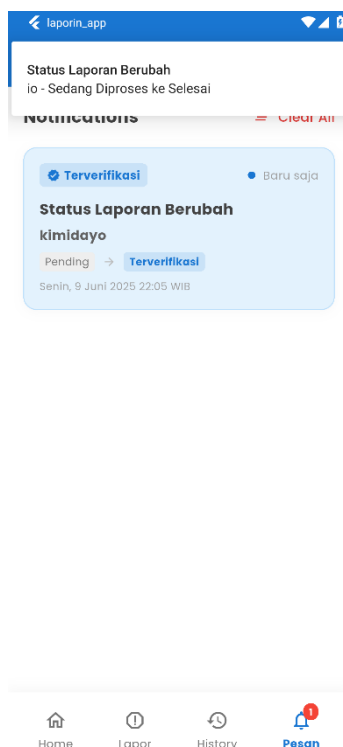
Photo Evidence



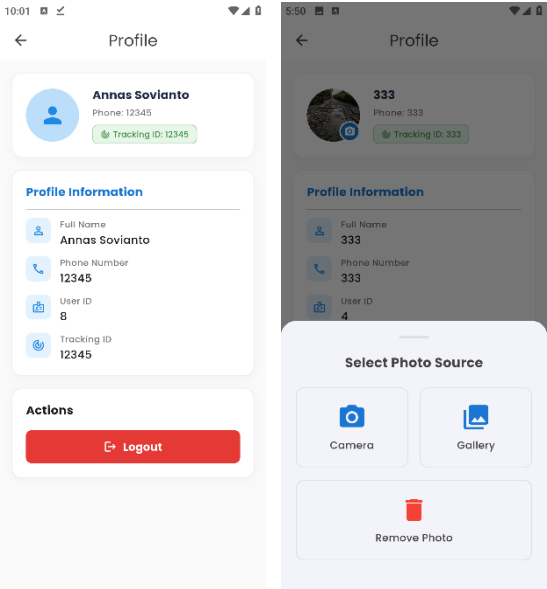
4. History User



5. Notification User



6. Profile Page



BAB III

PENGUJIAN

3.1 Metode Testing

Metode pengujian yang digunakan adalah Manual Testing dengan pendekatan Black Box. Pengujian ini berfokus pada Functional Testing dan Usability Testing di level System Testing.

1. Manual Testing: Pengujian dilakukan secara manual oleh penguji tanpa menggunakan alat otomatisasi.
2. Black Box Testing: Pengujian dilakukan tanpa mengetahui struktur internal atau kode program aplikasi. Penguji hanya berfokus pada input dan output sistem.
3. Functional Testing: Pengujian untuk memastikan bahwa setiap fungsi aplikasi berjalan sesuai dengan spesifikasi yang telah ditentukan.
4. Usability Testing: Pengujian untuk mengevaluasi kemudahan penggunaan, efisiensi, dan kepuasan pengguna terhadap aplikasi.
5. System Testing: Pengujian yang dilakukan pada seluruh sistem aplikasi untuk memastikan bahwa semua komponen bekerja bersama dengan benar

3.2 Rencana Pengujian

Rencana pengujian ini mencakup langkah-langkah yang akan dilakukan untuk menguji fitur registrasi, login, interaksi pengguna, dan logout pada aplikasi "Lapor.In". Pengujian akan dilakukan oleh penguji yang berpengalaman dalam pengujian perangkat lunak.

1. Persiapan:
 - a. Menyiapkan lingkungan pengujian (perangkat mobile dengan sistem operasi Android dan iOS).
 - b. Memastikan aplikasi "Lapor.In" telah terinstal dengan benar.
 - c. Membuat akun pengujian.
2. Pelaksanaan Pengujian:
 - a. Mengikuti skenario pengujian yang telah ditentukan (lihat tabel di bawah).
 - b. Mencatat hasil pengujian (hasil aktual, status, catatan).
 - c. Melaporkan bug atau masalah yang ditemukan.

3. Analisis Hasil Pengujian:

- a. Menganalisis hasil pengujian untuk mengidentifikasi masalah atau area yang perlu ditingkatkan.
- b. Memverifikasi perbaikan bug atau masalah yang telah dilaporkan.

3.3 Tabel Hasil Pengujian

No	Fitur	Skenario	Harapan Hasil	Hasil Aktual	Status	Catatan
1	Registrasi Pengguna: Pengisian Data Diri	Memasukkan nama lengkap, nomor telepon, dan kata sandi yang valid.	Semua field validasi berhasil dilewati. Tombol "Next" aktif dan dapat ditekan.	Semua validasi berhasil dilewati, tombol "Next" aktif.	Lulus	Pastikan pesan validasi yang ditampilkan jelas dan informatif.
2	Registrasi Pengguna: Pengisian Data Diri	Memasukkan nama lengkap kosong.	Field nama lengkap menampilkan pesan error "Please enter your full name". Tombol "Next" tidak aktif.	Field nama lengkap menampilkan pesan error, tombol "Next" tidak aktif.	Lulus	Pastikan pesan error mudah dibaca dan dipahami.
3	Registrasi Pengguna: Verifikasi Nomor Telepon	Memasukkan kode verifikasi yang benar.	Pengguna berhasil terdaftar dan diarahkan ke halaman login.	Pengguna berhasil terdaftar dan diarahkan ke halaman login.	Lulus	Pastikan proses verifikasi cepat dan lancar.
4	Registrasi Pengguna: Verifikasi Nomor Telepon	Memasukkan kode verifikasi yang salah.	Sistem menampilkan pesan kesalahan bahwa kode verifikasi salah.	Sistem menampilkan pesan kesalahan bahwa kode verifikasi salah.	Lulus	Pastikan pesan kesalahan jelas dan memberikan petunjuk untuk memperbaiki.
5	Registrasi Pengguna: Verifikasi Nomor Telepon	Tidak memasukkan kode verifikasi.	Sistem menampilkan pesan kesalahan bahwa kode verifikasi harus diisi.	Sistem menampilkan pesan kesalahan bahwa kode verifikasi harus diisi.	Lulus	Pastikan pesan kesalahan jelas dan memberikan petunjuk untuk memperbaiki.
6	Login Pengguna	Memasukkan nomor telepon dan kata sandi yang valid.	Pengguna berhasil login dan diarahkan ke halaman utama (views/navigation.dart).	Pengguna berhasil login dan diarahkan ke halaman utama.	Lulus	Pastikan transisi ke halaman utama mulus dan cepat.
7	Login Pengguna	Memasukkan nomor telepon atau kata sandi yang tidak valid.	Sistem menampilkan pesan kesalahan bahwa nomor telepon atau kata sandi salah.	Sistem menampilkan pesan kesalahan bahwa nomor	Lulus	Pastikan pesan kesalahan jelas dan memberikan

				telepon atau kata sandi salah.		petunjuk untuk memperbaiki.
8	Login Pengguna	Memasukkan nomor telepon yang belum terdaftar.	Sistem menampilkan pesan kesalahan bahwa nomor telepon belum terdaftar.	Sistem menampilkan pesan kesalahan bahwa nomor telepon belum terdaftar.	Lulus	Pastikan pesan kesalahan jelas dan memberikan petunjuk untuk mendaftar.
9	Pengajuan Laporan: Informasi Laporan	Mengisi judul, kategori, deskripsi, dan lokasi laporan.	Semua field validasi berhasil dilewati. Tombol "Next" aktif dan dapat ditekan.	Semua validasi berhasil dilewati, tombol "Next" aktif.	Lulus	Pastikan dropdown kategori dan lokasi berfungsi dengan baik.
10	Pengajuan Laporan: Informasi Laporan	Mengosongkan salah satu field wajib (judul, kategori, deskripsi, lokasi).	Sistem menampilkan pesan kesalahan yang sesuai pada field yang kosong. Tombol "Next" tidak aktif.	Sistem menampilkan pesan kesalahan, tombol "Next" tidak aktif.	Lulus	Pastikan pesan kesalahan jelas dan memberikan petunjuk untuk memperbaiki.
11	Pengajuan Laporan: Informasi Pelapor	Mengisi nama lengkap dan nomor telepon pelapor.	Semua field validasi berhasil dilewati. Tombol "Next" aktif dan dapat ditekan.	Semua validasi berhasil dilewati, tombol "Next" aktif.	Lulus	Pastikan format nomor telepon divalidasi dengan benar.
12	Pengajuan Laporan: Informasi Pelapor	Mengosongkan salah satu field wajib (nama lengkap, nomor telepon).	Sistem menampilkan pesan kesalahan yang sesuai pada field yang kosong. Tombol "Next" tidak aktif.	Sistem menampilkan pesan kesalahan, tombol "Next" tidak aktif.	Lulus	Pastikan pesan kesalahan jelas dan memberikan petunjuk untuk memperbaiki.
13	Pengajuan Laporan: Unggah Bukti	Mengunggah foto bukti laporan.	Foto berhasil diunggah dan ditampilkan sebagai preview. Tombol "Next" aktif dan dapat ditekan.	Foto berhasil diunggah dan ditampilkan, tombol "Next" aktif.	Lulus	Pastikan format foto yang didukung jelas dan proses unggah cepat.
14	Pengajuan Laporan: Unggah Bukti	Tidak mengunggah foto bukti laporan.	Sistem menampilkan pesan kesalahan bahwa foto bukti wajib diunggah. Tombol "Next" tidak aktif.	Sistem menampilkan pesan kesalahan, tombol "Next" tidak aktif.	Lulus	Pastikan pesan kesalahan jelas dan memberikan petunjuk

						untuk memperbaiki.
15	Pengajuan Laporan: Preview dan Kirim	Memeriksa semua informasi laporan dan mengirim laporan.	Laporan berhasil dikirim dan pengguna menerima notifikasi. Pengguna diarahkan kembali ke halaman utama.	Laporan berhasil dikirim, notifikasi diterima, pengguna diarahkan ke halaman utama.	Lulus	Pastikan semua informasi ditampilkan dengan benar dan proses pengiriman cepat.
16	Pengajuan Laporan: Pembatalan via Shake Detection	Menggoyangkan perangkat (Shake Detection).	Muncul dialog konfirmasi pembatalan laporan.	Muncul dialog konfirmasi pembatalan laporan.	Lulus	Sensitivitas sensor dapat disesuaikan.
17	Pengajuan Laporan: Penanganan Koneksi	Menekan tombol "Submit" tanpa koneksi internet.	Menampilkan pesan kesalahan tidak ada koneksi internet.	Menampilkan pesan kesalahan tidak ada koneksi internet.	Lulus	Pastikan penanganan koneksi internet berfungsi dengan baik.
18	Pelacakan Status Laporan: Daftar Laporan	Melihat daftar laporan yang pernah diajukan.	Daftar laporan ditampilkan dengan status yang benar.	Daftar laporan ditampilkan dengan status yang benar.	Lulus	Pastikan daftar laporan diurutkan dengan benar (misalnya, berdasarkan tanggal pengajuan).
19	Pelacakan Status Laporan: Detail Laporan	Melihat detail laporan tertentu.	Detail laporan ditampilkan dengan lengkap dan benar.	Detail laporan ditampilkan dengan lengkap dan benar.	Lulus	Pastikan semua informasi (judul, kategori, deskripsi, lokasi, status, foto) ditampilkan dengan benar.
20	Notifikasi: Penerimaan Notifikasi	Menerima notifikasi saat status laporan berubah.	Notifikasi diterima dengan tepat waktu dan menampilkan informasi yang relevan.	Notifikasi diterima dengan tepat waktu dan menampilkan informasi yang relevan.	Lulus	Pastikan notifikasi tidak duplikat dan dapat diklik untuk melihat detail laporan.
21	Notifikasi: Tampilan Daftar Notifikasi	Melihat daftar notifikasi.	Daftar notifikasi ditampilkan dengan benar.	Daftar notifikasi ditampilkan dengan benar.	Lulus	Pastikan notifikasi yang sudah dibaca ditandai

						dengan benar.
22	Lokasi: Pemilihan Lokasi Peta	Memilih lokasi menggunakan peta.	Lokasi berhasil dipilih dan ditampilkan pada field lokasi.	Lokasi berhasil dipilih dan ditampilkan pada field lokasi.	Lulus	Pastikan peta berfungsi dengan baik dan pencarian lokasi akurat.
23	Lokasi: Pencarian Lokasi	Mencari lokasi menggunakan fitur pencarian.	Lokasi yang dicari ditemukan dan ditampilkan pada peta.	Lokasi yang dicari ditemukan dan ditampilkan pada peta.	Lulus	Pastikan fitur pencarian berfungsi dengan baik dan memberikan hasil yang relevan.
24	Logout Pengguna	Melakukan logout dari aplikasi.	Pengguna berhasil logout dan diarahkan ke halaman login.	Pengguna berhasil logout dan diarahkan ke halaman login.	Lulus	Pastikan semua data sesi pengguna dihapus dengan benar.

BAB IV

PENUTUP

5.1 Kesimpulan

Pengembangan aplikasi "Lapor.In" sebagai platform pengaduan masyarakat berbasis mobile telah berhasil mencapai tujuan yang ditetapkan. Aplikasi ini menyediakan solusi yang inovatif, efisien, dan transparan bagi masyarakat untuk menyampaikan laporan atau keluhan kepada pihak berwenang.

Fitur-fitur utama seperti registrasi dan login pengguna, pengajuan laporan dengan lampiran bukti, pelacakan status laporan, notifikasi, dan integrasi dengan layanan lokasi telah diimplementasikan dengan baik. Penggunaan teknologi Flutter memungkinkan aplikasi untuk berjalan di berbagai platform (Android dan iOS) dengan antarmuka pengguna yang responsif dan menarik.

Pemanfaatan penyimpanan data lokal Hive memastikan ketersediaan data dan kinerja aplikasi yang optimal, bahkan dalam kondisi jaringan yang tidak stabil. Integrasi dengan API eksternal, seperti Google Maps API, memperkaya fungsionalitas aplikasi dan memberikan nilai tambah bagi pengguna.

Melalui pengujian yang komprehensif, aplikasi "Lapor.In" telah terbukti memenuhi kebutuhan fungsional dan non-fungsional yang telah ditetapkan. Aplikasi ini diharapkan dapat meningkatkan partisipasi masyarakat dalam pembangunan, mempercepat penyelesaian masalah, dan mewujudkan tata pemerintahan yang lebih baik.

5.2 Saran

Pengembangan aplikasi "Lapor.In" di masa mendatang dapat dipertimbangkan untuk meningkatkan fungsionalitas, kinerja, dan keamanan aplikasi. Pertama, penambahan fitur umpan balik dari masyarakat terhadap tindak lanjut laporan dapat meningkatkan akuntabilitas dan transparansi. Kedua, integrasi sistem pembayaran untuk donasi atau biaya administrasi (jika diperlukan) dapat memberikan sumber pendanaan tambahan untuk pengembangan aplikasi. Ketiga, pengembangan fitur pelaporan anonim dapat mendorong masyarakat untuk melaporkan masalah tanpa takut akan identifikasi. Keempat, optimasi kode dan database dapat meningkatkan kecepatan dan efisiensi aplikasi, serta mengurangi ukuran aplikasi untuk memudahkan pengunduhan dan instalasi. Kelima, penerapan enkripsi

data dan otentikasi yang lebih kuat dapat meningkatkan keamanan aplikasi dan melindungi data pengguna dari akses yang tidak sah. Terakhir, pengujian usabilitas dengan melibatkan pengguna nyata dapat memberikan umpan balik yang berharga untuk meningkatkan kemudahan penggunaan aplikasi.

DAFTAR PUSTAKA

- [1] Jurusan Informatika, "Modul Praktikum Teknologi dan Pemrograman Mobile," Universitas Pembangunan Nasional "Veteran" Yogyakarta, 2025.
- [2] Flutter Documentation. (n.d.). Introduction to Flutter. Diakses dari <https://flutter.dev/docs>
- [3] Hive Documentation. (n.d.). Hive: Lightweight and blazing fast key-value database. Diakses dari <https://docs.hivedb.dev/>
- [4] Provider Package. (n.d.). Provider: A simple way to access data in Flutter. Diakses dari <https://pub.dev/packages/provider>
- [5] HTTP Package. (n.d.). http | Dart Package. Diakses dari <https://pub.dev/packages/http>
- [6] Shared Preferences Package. (n.d.). shared_preferences | Dart Package. Diakses dari https://pub.dev/packages/shared_preferences
- [7] Geolocator Package. (n.d.). geolocator | Dart Package. Diakses dari <https://pub.dev/packages/geolocator>
- [8] Flutter Local Notifications Package. (n.d.). flutter_local_notifications | Dart Package. Diakses dari https://pub.dev/packages/flutter_local_notifications
- [9] Sensors Plus Package. (n.d.). sensors_plus | Dart Package. Diakses dari https://pub.dev/packages/sensors_plus
- [10] Google Maps API Documentation. (n.d.). Diakses dari <https://developers.google.com/maps>

LAMPIRAN

- [1] Github Repository:
<https://github.com/anndeviant/Lapor.In-mobile/tree/app-v1>
- [2] Spreadsheet Pengujian APP:
<https://docs.google.com/spreadsheets/d/1-ahBGOW-rQjm9tS27U6VBJY-JEs8vqrdG7NSYqvJRkI/edit?usp=sharing>
- [3] PPT Presentasi:
<https://www.canva.com/design/DAGp3x2Fp8A/198Vv8ik7H7L0Pt8OyWpYg/edit>